Security Architecture

The security architecture of StockWise is designed using a "Defense in Depth" strategy, ensuring that sensitive business data is protected at the transport, application, and persistence layers. Our security model integrates modern industry standards to guarantee system reliability and user data privacy within our monolithic architecture.

2.1 Authentication & Authorization (Session-Based Security) We utilize **Session-based Authentication** via **Spring Security** to manage user sessions.

- **Authentication Mechanism:** Upon successful login, the server creates an `HttpSession` and issues a secure, HTTP-only cookie to the client's browser. This cookie identifies the user session, eliminating the need for client-side token storage.
- **Role-Based Access Control (RBAC):** We implement granular authorization using Spring Security's role-based hierarchy. Access to both URL paths (e.g., `/admin/**`) and UI components (via Thymeleaf security dialect) is restricted, ensuring that users can only interact with functionality permitted by their role.

2.2 Data Protection & Integrity The system ensures that user data and business metrics remain immutable and secure at the persistence layer.

- **Credential Hashing:** User passwords are never stored in plain text. We utilize the **BCrypt hashing algorithm** with a strong, randomized salt to ensure credentials cannot be reversed.
- **Injection Prevention:** By leveraging **Spring Data JPA's** built-in parameter binding, we automatically mitigate the risk of SQL Injection attacks. Every query is treated as a parameterized command, ensuring input data cannot manipulate the underlying PostgreSQL database.

2.3 Transport Security & Network Hardening Data in transit between the user's browser and the server is protected against interception.

- **TLS/SSL Encryption:** All communication is mandated via **HTTPS/TLS**, ensuring that traffic is encrypted and protected from man-in-the-middle (MITM) attacks.
- **CSRF Protection:** Since we are using Thymeleaf, we enable **Cross-Site Request Forgery (CSRF) protection** by default. Spring Security automatically generates a unique CSRF token for every state-changing form submission, ensuring that only requests originating from our own application are processed.

2.4 Application Security

- **Input Validation:** We employ **Jakarta Bean Validation** (JSR 380) on all form data objects (DTOs) before processing. This sanitizes inputs at the controller level, preventing malformed data and mitigating potential Cross-Site Scripting (XSS) risks.
- **Session Management:** Our session configuration includes time-based inactivity timeouts and strict cookie attributes (Secure, HttpOnly) to prevent session hijacking and protect users from unauthorized access in shared environments.